

UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Imaxe pendente: figuras/logo_usc.eps

ESCOLA TÉCNICA SUPERIOR DE ENXEÑARÍA

Título do Traballo de Fin de Grao

Autor/a:
Gabriel Novo Vila

Titor:
Pablo García Tahoces

Grao en Enxeñaría Informática

Marzo 2026

Traballo de Fin de Grao presentado na Escola Técnica Superior de Enxeñaría
da Universidade de Santiago de Compostela para a obtención do Grao en
Enxeñaría Informática

Imaxe pendente: `figuras/logo_usc.eps`

D. (Pablo García Tahoces), Profesor/a do Departamento de Electrónica e Computación da Universidade de Santiago de Compostela

INFORMAN:

Que a presente memoria, titulada *Simulador de Máquina Enigma M3*, presentada por **D. Gabriel Novo Vila** para superar os créditos correspondentes ao Traballo de Fin de Grao da titulación de Grao en Enxeñaría Informática, realizouse baixo nosa titoría no Departamento de Electrónica e Computación da Universidade de Santiago de Compostela.

E para que así conste aos efectos oportunos, expiden o presente informe en Santiago de Compostela, a (Data):

Titor/a,

Alumno/a,

Pablo García Tahoces Gabriel Novo Vila

Agradecimentos

Se se quere pór algún agradecemento, este vai aquí.

Resumo

La máquina Enigma. ¿Qué es? ¿Quién la construyó? ¿Cuándo y por qué fue importante? Este trabajo se planteó sobre la base que forman estas preguntas, con el objetivo de acercar el funcionamiento y la historia de la máquina a personas que no necesitan disponer de conocimientos previos ni de criptografía ni de la máquina.

Para ello, se desarrolló un simulador funcional de una máquina Enigma M3. El sistema permite configurar sus ajustes internos, así como cifrar y descifrar mensajes observando los pasos seguidos durante el proceso. La aplicación también incluye opciones de accesibilidad y un historial de las transformaciones realizadas.

El proyecto está dividido en dos partes principales. El backend, desarrollado en Java con Spring, contiene la lógica de la máquina y ofrece sus funcionalidades mediante una API REST. El frontend, implementado con React, proporciona una interfaz web inspirada en la estética de la época y permite interactuar visualmente con los componentes simulados. Además, la aplicación dispone de inicio de sesión mediante cuentas de Google para ofrecer funciones asociadas al usuario.

El simulador se complementa con contenido educativo sobre los componentes de Enigma, su contexto histórico y el trabajo realizado para descifrarla. Finalmente, se proponen varios retos de descifrado en los que el usuario debe aplicar lo aprendido para descubrir un mensaje secreto enviado por un misterioso remitente.

Memoria tipo B – Índice xeral

1. Introducción	1
1.1. Objetivos generales	1
1.2. Descripción del sistema	3
1.3. Elección de las tecnologías	3
1.4. Organización de la memoria	3
2. La máquina Enigma	5
2.1. ¿Qué es la máquina Enigma?	5
2.1.1. Historia	5
2.1.2. Funcionamiento	8
2.2. Contexto histórico	10
2.3. Historia del descifrado de la máquina	11
3. Especificación de Requisitos	13
3.1. Requisitos funcionales	13
3.2. Requisitos no funcionales	15
3.3. Información almacenada por el sistema	16
3.3.1. Información guardada en base de datos	16
3.3.2. Información mantenida en memoria	17
3.3.3. Información guardada en disco	18
4. Deseño	19
5. Probas	21
6. Exemplos (eliminar capítulo na versión final)	23
6.1. Un exemplo de sección	23
6.1.1. Un exemplo de subsección	23
6.1.2. Otro exemplo de subsección	23
6.2. Exemplos de figuras e cadros	24
6.3. Exemplos de referencias á bibliografía	24
6.4. Exemplos de enumeracións	24
7. Conclusións e posibles ampliacións	27

A. Manuais técnicos	29
B. Manuais de usuario	31
C. Licenza	33
Bibliografía	35

Índice de figuras

6.1. Esta é a figura de tal e tal.	24
--	----

Índice de cadros

6.1. Esta é a táboa de tal e cal.	24
---	----

Capítulo 1

Introducción

La máquina Enigma ocupa un lugar destacado tanto en la historia de la criptografía como en la de la informática. Su utilización durante la Segunda Guerra Mundial, la complejidad de su funcionamiento y el esfuerzo colectivo necesario para descifrar sus comunicaciones la convierten en un caso especialmente interesante para comprender cómo la tecnología puede influir en acontecimientos históricos. Sin embargo, explicar su funcionamiento únicamente mediante descripciones teóricas puede resultar difícil para una persona que no disponga de conocimientos previos sobre criptografía.

El presente Trabajo de Fin de Grado tiene como objetivo principal desarrollar una aplicación web educativa que simule una máquina Enigma M3 y permita comprenderla mediante la interacción directa con sus componentes. El usuario puede configurar la máquina, cifrar y descifrar letras, observar los pasos seguidos durante el proceso y consultar explicaciones sobre su funcionamiento y su contexto histórico. Asimismo, se incluyen retos de descifrado que permiten aplicar de forma práctica los conocimientos adquiridos a lo largo de la página, pues existen secciones dedicadas tanto a la historia como al funcionamiento interno de la máquina.

1.1. Objetivos generales

El desarrollo del proyecto parte de los siguientes objetivos generales:

Por un lado, desde el backend del proyecto, lo que buscamos es reproducir el comportamiento de una máquina Enigma M3, centrándose en todas las posibilidades que nos presenta una: desde cambiar un rotor hasta cifrar una letra u obtener los pasos seguidos para este cifrado.

Por otro, buscamos, también, ofrecer una interfaz clara y visual que permita modificar la dicha configuración de la máquina y observar, con detalle, el resultado de cada operación realizada con esta simulación. Este frontend no solo actuaría como portal entre el usuario y el backend, sinoq que también nos provee de

opciones de accesibilidad para que todo tipo de persona pueda disfrutar la página: diferencias entre el modo claro y oscuro, modo para daltónicos y distintos tamaños de letra para que nadie se sienta excluido.

Tendría, también, secciones de historia o de funcionamiento, de forma que un usuario, sin saber nada de criptografía pueda aprender los orígenes de la misma y las funcionalidades provistas por cada pieza de la máquina. Consideramos necesaria la comprensión de la historia detrás de la máquina, pues, al final del día, es solamente una máquina más. La gente la tiene totalmente demonizada por el perfil de la gente que la utilizaba. Este trabajo no pretende ser ni político ni social, no es más que una demostración de hasta dónde puede llegar la creación de una persona y la prueba de que se puede separar la herramienta de la persona que la use.

Por supuesto, algunos de los objetivos propuestos también son técnicos:

- Diseñar una arquitectura que separe la lógica de simulación de la interfaz y permita mantener y ampliar el sistema es una buena práctica, de forma que no sea tan centralizado. Si falla la interfaz, siempre tienes la posibilidad de acceder a través de línea de comandos a la aplicación. Si falla el código, la interfaz no será capaz de cifrar, pero sí que podrás abrir las secciones de historia y funcionamiento para entender cómo funciona.
- Desarrollar una API REST que permita crear, configurar y eliminar máquinas, además de cifrar y descifrar letras, gestionar las conexiones del panel de cables y modificar los ajustes internos de los rotores. De esta forma, la lógica principal queda accesible de una manera ordenada y reutilizable.
- Mantener el estado de cada máquina durante la sesión, incluyendo rotores, posiciones actuales, reflector y cables conectados. Esto permite que la simulación no sea una simple transformación aislada de letras, sino un proceso continuo donde cada pulsación afecta al estado posterior de la máquina.
- Implementar un sistema de trazabilidad de las operaciones, de manera que el usuario pueda consultar los pasos seguidos por cada letra cifrada o descifrada. Este punto es importante porque el proyecto no solo busca devolver un resultado, sino enseñar cómo se llega a él.
- Incorporar autenticación mediante Google para asociar ciertas funcionalidades al usuario, como el historial de sesión o el historial guardado. Con esto se añade una capa de uso más cercana a una aplicación real y no solo a una demostración local.

1.2. Descripción del sistema

La aplicación se divide en dos partes principales. El backend, desarrollado en Java con Spring, contiene la lógica de la máquina Enigma y expone sus operaciones mediante una API REST. Esta parte se encarga de crear y configurar máquinas, gestionar sus conexiones y procesar las letras durante el cifrado y el descifrado.

El frontend, desarrollado con React, proporciona la interfaz web con la que interactúa el usuario. Además del simulador, incluye secciones dedicadas al funcionamiento de los componentes, la historia de Enigma, el libro de codificación y distintos retos. La aplicación también permite iniciar sesión mediante una cuenta de Google y conservar un historial asociado al usuario.

1.3. Elección de las tecnologías

A lo largo de la etapa universitaria se presentaron distintas tecnologías y múltiples formas de resolver un mismo problema. Una decisión técnica no debe valorar únicamente la máxima eficiencia posible, sino también el contexto del proyecto, la experiencia previa y la capacidad de desarrollar una solución comprensible y mantenible.

Por este motivo se escogieron Java y Spring para el backend. Estas tecnologías permitieron trabajar con herramientas conocidas y construir una API estructurada en la que la lógica de la máquina estuviese claramente separada del resto del sistema. Para el frontend se eligió JavaScript con React, debido a su capacidad para crear interfaces interactivas y actualizar visualmente el estado de los distintos componentes.

El objetivo no es presentar la aplicación más revolucionaria ni afirmar que las tecnologías elegidas sean las únicas adecuadas. El proyecto busca demostrar que, cuando las decisiones se adaptan correctamente al entorno de desarrollo, es posible construir una aplicación funcional, educativa y ampliable utilizando herramientas con las que el desarrollador se siente cómodo trabajando.

1.4. Organización de la memoria

Esta memoria comienza con una explicación de la máquina Enigma lo más cercana al usuario posible, navegando a través de los componentes de la máquina, pasando por la historia de la misma, uniendo las complicaciones y las razones detrás del éxito de la misma, hasta llegar al evento que puso fin a la guerra: la decodificación de la máquina. Luego, se hablará de la especificación de los requisitos más relevantes del sistema. A continuación, se describe su diseño, la división entre componentes y las decisiones tomadas durante la implementación.

Posteriormente, se presenta el plan de pruebas utilizado para comprobar el funcionamiento de la aplicación. Finalmente, se exponen las conclusiones obtenidas y las posibles ampliaciones futuras del proyecto.

Capítulo 2

La máquina Enigma

La máquina Enigma es uno de los dispositivos criptográficos más conocidos de la historia y constituye un símbolo de la importancia que la criptografía tuvo durante la primera mitad del siglo XX. Su utilización por parte de Alemania durante la Segunda Guerra Mundial y el posterior esfuerzo realizado para descifrar sus mensajes la han convertido en una pieza clave tanto en la historia de la tecnología como en la de los conflictos bélicos.

En este capítulo se presenta una visión general de la máquina Enigma, comenzando por una breve introducción a la evolución histórica de la criptografía y a algunos de los métodos de cifrado que precedieron a su aparición. Posteriormente se explica el funcionamiento interno de la máquina, sus componentes principales y el contexto histórico en el que fue utilizada. Finalmente, se aborda el proceso de descifrado de sus comunicaciones y la contribución de las personas que participaron en esta tarea.

2.1. ¿Qué es la máquina Enigma?

2.1.1. Historia

Orígenes de la criptografía

La respuesta simple es que la máquina Enigma es una máquina de cifrado por rotores, pero esa no sería una buena respuesta. Para poder responder adecuadamente a esta pregunta tenemos que preguntarnos "¿Qué es el cifrado?"

Nos referimos a cifrar cuando hablamos del proceso de codificación de la información. Codificar la información, en palabras simples, es convertir el mensaje que queremos transmitir en otro. Este segundo mensaje puede tener sentido o puede que no lo tenga, lo normal es que no lo tenga. A la ciencia que estudia el cifrado de mensajes se le llama Criptografía. Si queremos hablar de esta ciencia, nos remontaremos al siglo V aC, nos remontamos a la guerra del Peloponeso.

La criptografía se cree que nace en la guerra de Esparta y Atenas. La comunicación entre los distintos aliados de un bando siempre fue algo de suma importancia en una contienda bélica y fue entonces cuando pensaron que la guerra podría dar una vuelta si, el mensaje, aunque fuera interceptado, solo lo pudieran entender ellos. Esto fue clave en el avance de la guerra por parte del bando griego, pues lograron crear el primer método registrado de cifrado.

La escítala

Los griegos usaban la llamada "Escítala", que constaba de un palo, rodillo o escalta que rodeaban de una tira de papel, ambos elementos de un tamaño adecuado y preciso. Existe un debate en la comunidad que menciona si este método de cifrado verdaderamente fue fruto de esta sociedad o si ya existía en otras polis. Como no es claro, lo trataremos como si hubiese surgido en la mencionada.

La escítala era un método sencillo, no requería de memorización ni de algoritmos complejos. Si tenías 2 varas del mismo tamaño, tenías el mensaje. Usualmente se escogían varas hexagonales u octogonales por sus caras planas. Alrededor, se enrollaba la tira de papel y se escribía sobre ella, una línea en cada cara de la vara. Cuando desenrollabas la cinta, tenías una tira de papel con letras aparentemente aleatorias en ella. Así, solo quien tuviera una escalta del mismo tamaño podía descifrar el mensaje, volviendo a enrollar la tira sobre la misma.

Si nos fijamos en el contexto histórico que este tenía detrás, podemos decir que no acababa de ser el mejor método de cifrado. En esta época todavía no existían los métodos de puntuación y no se utilizaban espacios para separar las palabras. Por si fuera poco, todas las palabras se escribían en mayúscula, por lo que era responsabilidad del lector identificar dónde empezaban y terminaban las palabras. Todo esto dificultaba la lectura del mensaje, aún siendo descifrado. Aristóteles decía que este método, además, era fácilmente descifrable: si alguien probaba con distintos grosores de vara y distintas formas, no había tantas posibilidades. De todas formas, esto requería tiempo y todos sabemos que, en la guerra, cada segundo cuenta.

Cifrado César

Cuanto más avanzamos en el tiempo, más tipos de cifrado nos podemos encontrar. El cifrado César fue el ideado por los romanos en el siglo I aC. Este era, de igual forma que el anterior, simple. Pero no requería de ningún artilugio, solo hacía falta conocer un número.

El cifrado César consistía en coger el alfabeto y moverlo 3 letras a la derecha. Así, la A se convertía en la D, la B en la E, la C en la F y así progresivamente hasta volver a la A. Esto ocasiona mensajes aparentemente sin sentido, pero sigue manteniendo los patrones del habla. Si una palabra es corta, véase artículos como "el." "la" en castellano, seguirán siendo así. Como son palabras muy comunes, es

sencillo asociar palabras, esto asociará letras y poco a poco nos dará el alfabeto entero. También, si una letra es muy común en el lenguaje, lo será su letra asociada bajo este cifrado.

El número que se dijo que hacía falta conocer era el desplazamiento. Se mencionó que se movían las letras 3 posiciones a la derecha, pero esto fue variando con el paso del tiempo.

La máquina Enigma

Ahora que hablamos de distintas formas de cifrado, podemos pasar a entender mejor qué hace la máquina Enigma y cómo funciona.

En el cifrado César, se formaba un nuevo abecedario. Podemos pensarlo como que se hacía una asociación 1 a 1 con las letras de ambos. Así, teníamos ABC-DEF... y DEFGHI... Donde la primera letra del primer abecedario se sustituía con la primera del segundo, la segunda letra con la segunda y así hasta terminar el abecedario. Esta es la base de la máquina Enigma.

Una vez entendido lo anterior, podemos pasar a responder la pregunta "¿Qué es la máquina Enigma?". La máquina Enigma es una máquina de cifrado por rotores. Un rotor es, en resumidas cuentas, una pieza que rota. Se explicó el funcionamiento de esta pieza en la sección "¿Cómo funciona?". Lo importante ahora es entender que cada rotor tiene un abecedario nuevo, como si tuviéramos un pequeño cifrado César metido en cada pieza. Este abecedario no sigue un patrón como los anteriores, no son 3 posiciones a la derecha, sino que es una selección sin patrón del orden de las letras. De igual forma, se hace una asociación 1 a 1 para las letras.

Hablar de la "máquina Enigma" como entidad universal es un error de concepto. Hay múltiples versiones de la misma, por ende, hay múltiples máquinas Enigma. El simulador montado en esta página es la versión M3, que es la primera versión modificada por la armada. Esta, como nuevas adiciones, incluía los tipos IV y V de rotores y los tipos B y C del reflector

Más tarde en la guerra, en 1942 se creó la versión M4 de la máquina, que incluía nuevos tipos de rotores, nuevos reflectores y un rotor adicional. Ahora, en vez de 3 rotores simultáneamente, tenía 4. Este rotor extra le daba una complejidad extra a la decodificación de los británicos de unas 50 a 100 veces, es esta la razón detrás de la adición del nuevo rotor. De todas formas, un error en el cifrado hizo que el descubrimiento del funcionamiento de este nuevo rotor fuera posible, por lo que, realmente, no tuvo el efecto deseado.

2.1.2. Funcionamiento

Rotores

Cada rotor contiene un tipo, un alfabeto propio, una posición inicial, una actual y una posición de cambio. Dependiendo del tipo de máquina, hay unos tipos de rotor u otros y este influye en su alfabeto y en la posición de cambio.

La función del rotor es cifrar una letra en base al alfabeto que contiene. A este le llega una entrada, que es una letra y devolverá una distinta. Cada letra de entrada tiene asociada otra de salida, de forma que podemos tener $A \rightarrow H$, $B \rightarrow D$, etc.

Ahora vamos a complicarlo un poco más, los rotores de las máquinas enigma giran para cambiar de posición. Cada vez que pulsas una letra en el teclado, el que llamaremos el primer rotor (el situado más a la derecha), gira. Si, por ejemplo, los rotores están en una posición A A A (La letra asociada a la posición del primer rotor es A, la del segundo, es A y la del tercero, es A), si cifras o descifras una letra, pasarán a A A B. Luego a A A C y así continuamente. Dependiendo del tipo de rotor, este tendrá una posición de cambio. Si la posición de cambio es H, al llegar a H, cambiará el siguiente rotor. Así, tendríamos la siguiente secuencia: A A F $\rightarrow$ A A G $\rightarrow$ A B H $\rightarrow$ A B I, cuando vuelva a llegar a H, volverá a cambiar el siguiente.

¿Cuál es el efecto de estas posiciones? Pues bien, además de estas, existe una posición inicial para cada rotor, que nos indica el desplazamiento para el alfabeto. La forma más fácil de entenderlo es pasando las letras a números. Imaginemos que al rotor llega la letra 1 (A). Este rotor está en una posición 3 (C) y su posición inicial era 2 (B). Como hay un cambio de 1 posición entre la actual y la inicial ($3-2=1$), el rotor actúa como si, en lugar de llegar la letra A, llegara la letra B ($1+1=2$). Se devuelve la letra asociada a la B. Imaginemos que es la M la letra asociada a la B. Ahora, se le restaría la diferencia que antes se sumó. Antes sumamos la letra que llegó ($A+1=B$), ahora lo restamos ($M-1=N$), en este ejemplo, si entrara una A, devolvería una N. Este ejemplo era de cifrado. En un caso de descifrado, primero se resta y luego se suma.

Reflector

El reflector es un elemento que el lector puede pensar como un rotor muy simple, como uno que no gira. Los únicos elementos de un reflector son su tipo y su alfabeto, dependiente del tipo del mismo.

Un reflector también tiene una señal de entrada y otra de salida. Esto significa que recibe una letra y, dependiendo del alfabeto del mismo, devolverá otra.

Para entenderlo bien, un reflector tiene un alfabeto como el de un rotor: En lugar de ser ABCDEF... podría ser PLTHYA..., en este ejemplo, si llega una A, devuelve una P, si llega una B, devuelve una L, etc.

La cualidad que no comparte con el rotor, además de que es un elemento fijo que no tiene posiciones, es que las letras están “unidas”. En el ejemplo anterior, si llega una A, devuelve una P. Si llegara una P, este devolvería una A. Esto no tiene por qué pasar en un rotor, pero es una cualidad necesaria en el reflector.

Teclado

En anteriores elementos comentamos que, dependiendo de la versión de la máquina, podíamos tener unos tipos u otros, esto no ocurre con el teclado. Siempre es el mismo.

El teclado de la Máquina Enigma está basado en un QWERTZ. Para quien no esté familiarizado con los teclados, en la mayoría de países del mundo se usa el teclado QWERTY. Este nombre hace referencia a las 6 primeras letras del teclado. El primero mencionado es el utilizado en países con lenguas germánicas, pues la Z es una letra más común que la Y.

Se mencionó que está “basado” en un QWERTZ y no que “sea” un QWERTZ por las ligeras diferencias que este presenta. Tiene una distribución muy curiosa de las teclas. Al no haber símbolos ni números, quedarían huecos, por lo que era necesaria una distribución algo distinta a la par que cómoda y usual para los usuarios.

Desde este componente se decide qué letra va a ser la cifrada, es el comienzo de la señal que se enviará a través de los rotores, el reflector y el espacio de conexiones.

Espacio de conexiones

Toda máquina Enigma contiene un espacio de conexiones cercano al teclado.

Este espacio de conexiones consiste en una serie de entradas disponibles donde cada una representa una letra. Está pensado para conectar letras entre ellas.

Cuando enviamos una letra con el teclado, antes de llegar a los rotores, pasa por el espacio de conexiones, es el primer paso. Es sencillo: si, con el teclado, pulsamos la A, primero mirará en este espacio si hay una conexión con la A; si no la hay, al primer rotor le llegará una A como entrada. En caso de que, por ejemplo, exista una conexión entre la entrada A y la entrada B, al rotor le llegará una B. Esta conexión funciona en ambos sentidos, es decir, si mantenemos esa conexión A-B e introducimos una B, al primer rotor llegará una A.

Salida

Las máquinas Enigma originales utilizaban un panel de bombillas. Tras completar el recorrido, se iluminaba la bombilla correspondiente a la letra obtenida.

La letra iluminada es el resultado final. Para formar un mensaje completo, el operador debía anotar cada salida y repetir el proceso letra por letra.

Como el recorrido es reversible cuando la configuración coincide, la salida cifrada puede introducirse de nuevo para recuperar el texto original.

2.2. Contexto histórico

La Máquina Enigma fue la máquina utilizada por las Potencias del Eje en la Segunda Guerra Mundial para cifrar sus comunicaciones de una manera segura. Para entender la Máquina Enigma y sus orígenes tenemos que remontarnos al comienzo del siglo XX, antes de que el partido nazi alcanzara el poder.

Arthur Scherbius, nacido en 1878 en Fráncfort del Meno y graduado en ingeniería eléctrica tanto en la Universidad Técnica de Munich como en la Universidad de Hannover, decide fundar una empresa bajo el nombre de Scherbius & Ritter. De esta forma, hizo múltiples inventos, entre los que estaba cierta máquina basada en ruedas cableadas giratorias, hoy día conocida como máquina de rotores. Los modelos creados se apodaban con letras. El primero fue el modelo A, el segundo el B y el tercero y último que creó, el modelo C. La máquina Enigma utilizada durante la guerra era del tamaño de una máquina de escribir y, de hecho, se parecía mucho en cuestiones como el teclado o la forma general de la máquina. Estos primeros modelos se asemejaban más a una máquina registradora.

En 1918, Arthur patenta su máquina, pensada originalmente para actuar como cifradora para empresas importantes como bancos. En estos años nos enfrentamos a una situación de gran inestabilidad política y social. Es el nacimiento de la República de Weimar, el fin de la Gran Guerra. Ante esta situación, Scherbius dudaba de si realmente la patente de la máquina llegaría a dar frutos, la realidad era que nadie parecía mostrar interés ante esta máquina. Esto, si bien es correcto cuando hablamos del ciudadano promedio, no es de igual forma para el gobierno. El gobierno alemán estaba altamente interesado en una máquina codificadora plenamente funcional e indescifrable, pues un año antes, en 1917, se interceptó y decodificó el Telegrama de Zimmermann, lo que puso a Estados Unidos en el mapa y cambió el transcurso de la batalla.

En 1926, ya comercializada bajo el nombre "Enigma", fue adoptada por la Armada alemana y, unos años más tarde, en 1934, la versión M3 de la máquina fue adoptada por la armada naval. Para este entonces, Arthur Scherbius ya no estaba presente para ver el alcance de su invención, falleció en 1929 sin conocer la importancia que tuvo la existencia de su invento en una guerra que tampoco alcanzó a vislumbrar.

En 1939, a punto de empezar la guerra, se crea la máquina enigma M4, que será la que usarán los aliados de Alemania durante toda la guerra, hasta que Alan Turing y su equipo logren descifrarla.

2.3. Historia del descifrado de la máquina

El reto de descifrar Enigma

Tener una Máquina Enigma no era suficiente para poder leer los mensajes del bando alemán. Para cifrar y descifrar un mensaje era necesario que ambas máquinas estuvieran configuradas exactamente de la misma forma. Los operadores alemanes utilizaban libros de claves donde aparecían los ajustes correspondientes a cada día del mes, como los rotores que debían utilizar y sus posiciones iniciales. Esta configuración cambiaba diariamente, por lo que descubrir la clave de un día no solucionaba el problema para siempre: al día siguiente, el trabajo comenzaba de nuevo.

Estos libros eran, por tanto, información de enorme valor. La captura de material alemán, como máquinas, manuales o libros de códigos, permitió conocer mejor el funcionamiento de sus redes de comunicación y facilitó el trabajo de los equipos de descifrado. Aun así, seguía siendo necesario encontrar la configuración concreta utilizada en cada jornada entre una cantidad inmensa de posibilidades.

Bletchley Park

El principal centro británico dedicado al descifrado se encontraba en Bletchley Park, una propiedad situada al norte de Londres que durante la guerra pasó a ser conocida también como la Estación X. Allí se reunieron matemáticos, criptógrafos, lingüistas, ingenieros, jugadores de ajedrez, personas aficionadas a los crucigramas y muchos otros perfiles que podían aportar una manera distinta de enfrentarse al problema.

El descifrado de Enigma no consistía únicamente en observar una máquina y descubrir su funcionamiento. Primero se interceptaban los mensajes enviados por radio, después se buscaban posibles fragmentos del texto original y, finalmente, se probaban y descartaban configuraciones hasta encontrar una que tuviera sentido. Todo este proceso se realizaba bajo un secreto absoluto y de forma continua, mediante distintos turnos de trabajo.

Alan Turing y un mérito compartido

Cuando se habla del descifrado de Enigma, el nombre que más se repite es el de Alan Turing. Su mérito es indiscutible: fue una figura fundamental en Bletchley Park y participó en el diseño de la Bombe, una máquina electromecánica que permitía descartar rápidamente configuraciones incompatibles con un mensaje y reducir el número de posibilidades que debían comprobarse manualmente.

Sin embargo, presentar esta historia como el logro de una sola persona sería injusto. La Bombe británica partía del trabajo previo de los criptógrafos polacos, entre los que destacó Marian Rejewski. En su desarrollo también fueron esenciales las mejoras de Gordon Welchman y el trabajo de ingeniería de Harold Keen.

Junto a ellos participaron miles de personas encargadas de interceptar, organizar, estudiar y comprobar mensajes. Alan Turing merece ser recordado, pero también lo merece el equipo que hizo posible convertir sus ideas en resultados diarios.

Las descodificadoras invisibles

Las mujeres formaban aproximadamente tres cuartas partes del personal de Bletchley Park, aunque durante décadas su participación quedó oculta tras una historia contada principalmente a través de figuras masculinas. No se limitaron a realizar tareas auxiliares: operaron máquinas, analizaron mensajes y formaron parte directa de los equipos de criptografía.

Joan Clarke trabajó en el equipo de Alan Turing y llegó a ser subdirectora de la cabaña 8, dedicada a los mensajes de la armada alemana. Mavis Lever y Margaret Rock formaron parte del equipo dirigido por Dilly Knox y ayudaron a romper el código utilizado por el servicio de inteligencia alemán. Estos nombres representan a muchas otras mujeres cuyo trabajo permaneció en secreto y que también fueron responsables del éxito de Bletchley Park.

Descifrando Enigma

En 2014 se estrenó *The Imitation Game*, conocida en España como *Descifrando Enigma*. La película fue dirigida por Morten Tyldum, está basada en la biografía de Alan Turing escrita por Andrew Hodges y cuenta con Benedict Cumberbatch en el papel protagonista. Su estreno ayudó a que una gran cantidad de personas conocieran la historia de Turing y el trabajo realizado en Bletchley Park.

De todas formas, debe entenderse como una dramatización y no como un documental. La película modifica o simplifica ciertos acontecimientos y concentra gran parte del relato en la figura de Turing. Es una buena puerta de entrada a esta historia, pero el descifrado real de Enigma fue un esfuerzo colectivo mucho más amplio, construido sobre el trabajo de polacos, británicos, hombres y mujeres cuyos nombres no siempre aparecen en pantalla.

Capítulo 3

Especificación de Requisitos

Especificación dos requisitos máis relevantes do Sistema, xunto coa información que este debe almacenar e as interfaces con outros Sistemas, sexan hardware ou software, e outros requisitos (rendemento, seguridade, etc.).

3.1. Requisitos funcionales

Los requisitos funcionales describen las acciones que debe permitir realizar el sistema. En este proyecto se han considerado los siguientes:

- **RF-01. Crear una máquina Enigma M3.** El sistema debe permitir crear una instancia de una máquina Enigma M3 desde el backend, asignándole un identificador único y una configuración inicial.
- **RF-02. Configurar los rotores.** El sistema debe permitir modificar los tres rotores utilizados por la máquina, así como sus posiciones iniciales, las posiciones actuales y los ajustes del anillo.
- **RF-03. Configurar el reflector.** El sistema debe permitir alternar entre los reflectores disponibles en la simulación. En la interfaz se muestran como reflector B y reflector C, aunque internamente se representan mediante los valores 0 y 1.
- **RF-04. Cifrar letras.** El sistema debe permitir introducir una letra y obtener la letra cifrada correspondiente, aplicando el estado actual de rotores, reflector y panel de conexiones.
- **RF-05. Descifrar letras.** El sistema debe permitir descifrar letras usando el mismo flujo de procesamiento que el cifrado, pero desde el modo de descifrado seleccionado por el usuario.

- **RF-06. Avanzar los rotores tras cada operación.** Cada vez que se cifra o descifra una letra, el sistema debe actualizar la posición actual de los rotores de acuerdo con el mecanismo definido en la simulación.
- **RF-07. Gestionar el panel de conexiones.** El sistema debe permitir conectar y desconectar pares de letras. Una letra conectada debe transformarse en su pareja tanto antes como después de pasar por los rotores y el reflector.
- **RF-08. Mostrar la salida de la máquina.** La interfaz debe mostrar la letra resultante de cada operación y construir visualmente el mensaje introducido junto con su salida.
- **RF-09. Mostrar los pasos seguidos.** El sistema debe permitir consultar los pasos internos realizados durante el cifrado o descifrado de una letra, incluyendo el paso por cables, rotores y reflector.
- **RF-10. Mantener historial de sesión.** La aplicación debe conservar las últimas operaciones realizadas durante la sesión actual para que el usuario pueda revisarlas.
- **RF-11. Mantener historial asociado al usuario.** Cuando el usuario inicia sesión, la aplicación debe mantener un historial asociado a su cuenta en el navegador.
- **RF-12. Iniciar sesión con Google.** El sistema debe permitir autenticación mediante Google OAuth2, generando tokens de acceso y refresco para identificar al usuario.
- **RF-13. Cerrar sesión.** El usuario debe poder cerrar la sesión, eliminando los tokens almacenados en el navegador.
- **RF-14. Renovar el token de acceso.** El sistema debe permitir solicitar un nuevo token de acceso a partir de un token de refresco válido.
- **RF-15. Ofrecer opciones de accesibilidad.** La aplicación debe permitir activar modo claro, modo para daltónicos y modificar el tamaño de letra.
- **RF-16. Identificar conexiones en modo daltónico.** Cuando el modo para daltónicos está activado, las conexiones del panel deben mostrar identificadores adicionales para no depender únicamente del color.
- **RF-17. Navegar por las secciones informativas.** La aplicación debe permitir acceder a las secciones de funcionamiento, historia, libro de codificación y retos desde la barra de navegación.

- **RF-18. Mostrar contenido educativo sobre la máquina.** El sistema debe incluir explicaciones sobre los principales componentes de Enigma: teclado, panel de conexiones, rotores, reflector, salida y flujo completo.
- **RF-19. Mostrar retos de descifrado.** La aplicación debe presentar mensajes cifrados para que el usuario intente resolverlos utilizando la máquina.
- **RF-20. Mostrar pistas en los retos.** Cada reto debe poder mostrar una pista individual que oriente al usuario durante el descifrado.

3.2. Requisitos no funcionales

Los requisitos no funcionales establecen restricciones medibles sobre el comportamiento del sistema. Para esta aplicación se proponen los siguientes:

- **RNF-01. Tiempo de respuesta del cifrado y descifrado.** El backend debe devolver el resultado de cifrar o descifrar una letra en menos de 500 ms en ejecución local.
- **RNF-02. Actualización visual tras cada letra.** Tras pulsar una tecla, la interfaz debe actualizar la salida, los rotores y el historial visible en menos de 1 segundo.
- **RNF-03. Carga inicial.** La página principal debe estar disponible en menos de 3 segundos en un navegador moderno ejecutándose en local.
- **RNF-04. Límite del historial visible.** La aplicación debe mantener como máximo 60 operaciones en el historial de sesión y en el historial asociado al usuario.
- **RNF-05. Capacidad del panel de conexiones.** El panel debe permitir hasta 13 pares de conexiones, equivalentes a las 26 letras del alfabeto conectadas sin repetición.
- **RNF-06. Compatibilidad de navegadores.** La aplicación debe funcionar correctamente en versiones recientes de Google Chrome, Microsoft Edge y Mozilla Firefox.
- **RNF-07. Resoluciones soportadas.** La interfaz debe ser utilizable en pantallas de escritorio desde 1366 x 768 píxeles y en pantallas móviles desde 360 píxeles de ancho.
- **RNF-08. Cambio entre secciones.** La navegación entre las secciones principales debe completarse en menos de 1 segundo, al tratarse de una aplicación de una sola página.

- **RNF-09. Apertura de menús.** Los menús de cuenta, historial y accesibilidad deben abrirse o cerrarse en menos de 300 ms tras la interacción del usuario.
- **RNF-10. Tamaño máximo de respuesta.** Las respuestas de cifrado y descifrado deben mantenerse por debajo de 10 KB, incluyendo letra resultante, máquina actualizada y pasos seguidos.
- **RNF-11. Consistencia del estado.** Tras cada operación de cifrado o descifrado, las posiciones de los rotores mostradas en la interfaz deben coincidir con las posiciones almacenadas en el backend.
- **RNF-12. Persistencia de sesión autenticada.** La sesión debe mantenerse mientras los tokens almacenados en el navegador sigan siendo válidos.
- **RNF-13. Aislamiento del historial por usuario.** El historial guardado en el navegador debe separarse por cuenta autenticada, utilizando una clave distinta para cada correo electrónico.
- **RNF-14. Accesibilidad cromática.** En modo daltónico, la información del panel de conexiones no debe depender exclusivamente del color.
- **RNF-15. Ajuste del tamaño de letra.** La interfaz debe permitir modificar el tamaño de letra entre los límites definidos por la aplicación, manteniendo la página utilizable.
- **RNF-16. Disponibilidad en local.** El sistema debe poder ejecutarse localmente con el frontend en el puerto 3000 y el backend en el puerto 8082.
- **RNF-17. Recuperación ante fallo del backend.** Si el backend no está disponible al iniciar la página, la interfaz no debe quedar bloqueada y debe mostrar un estado informativo.

3.3. Información almacenada por el sistema

El sistema combina información gestionada por el backend con información mantenida por el frontend en el navegador. A continuación se separa según el lugar donde se almacena.

3.3.1. Información guardada en base de datos

El backend utiliza Spring Data JPA y un repositorio `CrudRepository` para almacenar objetos de tipo `M3DT0`. La configuración actual del proyecto utiliza H2 en memoria mediante la propiedad `spring.datasource.url=jdbc:h2:mem:testdb`.

Por tanto, aunque se usa una base de datos, los datos no permanecen tras reiniciar el servidor.

En la base de datos se guarda la información necesaria para reconstruir el estado de cada máquina Enigma M3:

- **Identificador de la máquina:** campo `id`, utilizado para localizar una máquina concreta desde los endpoints.
- **Rotores seleccionados:** lista `rotores`, donde se almacena el tipo de cada uno de los tres rotores.
- **Configuración del anillo:** lista `rotores_settings`, que guarda la letra asociada al ajuste del anillo de cada rotor.
- **Posiciones actuales de los rotores:** lista `rotores_posiciones`, que cambia al cifrar o descifrar letras y también puede modificarse desde la interfaz.
- **Cables del panel de conexiones:** lista `cables`, formada por pares de índices de letras. Cada cable se representa mediante los campos `a` y `b`.
- **Reflector seleccionado:** campo `reflector`, almacenado internamente como 0 o 1.

No se almacenan en la base de datos los textos educativos, los retos, las preferencias de accesibilidad, los tokens de autenticación ni el historial visible del usuario.

3.3.2. Información mantenida en memoria

Durante la ejecución, el backend crea objetos auxiliares en memoria para procesar cada operación. Entre ellos se encuentran la representación completa de la máquina M3, el alfabeto, el mapa temporal de cables, los rotores, el reflector y la lista de pasos generada durante el cifrado o descifrado de una letra. Esta información se usa durante la petición y después se transforma de nuevo en M3DTO para actualizar la base de datos.

En el frontend también existe estado mantenido en memoria mediante React. Este estado incluye, entre otros elementos, la máquina actualmente mostrada, la letra de salida, el historial de la sesión actual, los últimos pasos consultados, la página activa, el modo de cifrado o descifrado, el cable seleccionado temporalmente, la apertura de menús y paneles, y los valores actuales de accesibilidad.

Esta información se pierde al cerrar o recargar la página, salvo aquella que también se escribe en `localStorage`.

3.3.3. Información guardada en disco

El backend no guarda información propia de la aplicación en disco durante la ejecución normal. La base de datos H2 configurada es en memoria, por lo que las máquinas creadas desaparecen al reiniciar el servidor.

En el lado del cliente sí existe almacenamiento persistente en el navegador mediante `localStorage`. Este almacenamiento pertenece al navegador del usuario y se conserva en el equipo o dispositivo hasta que el usuario cierre sesión, borre los datos del navegador o la aplicación lo sobrescriba. Se guardan los siguientes datos:

- `accessToken`: token de acceso usado para identificar la sesión actual.
- `refreshToken`: token utilizado para solicitar un nuevo token de acceso.
- `enigma-history:<correo>`: historial asociado a la cuenta autenticada, limitado a 60 operaciones.
- `enigma-accessibility-light`: preferencia del modo claro.
- `enigma-accessibility-colorblind`: preferencia del modo para daltónicos.
- `enigma-accessibility-font-scale`: escala de tamaño de letra seleccionada por el usuario.

Por tanto, no existe persistencia en disco en el servidor para los datos de dominio de la aplicación. La única persistencia duradera que se realiza actualmente corresponde al almacenamiento local del navegador.

Capítulo 4

Deseño

Debe describirse como se realiza o Sistema, a división deste en diferentes compoñentes e a comunicación entre eles. Así mesmo, determinarase o equipamento hardware e software necesario, xustificando a súa elección no caso de que non fose un requisito previo. Debe achegarse a un nivel suficiente de detalle que permita comprender a totalidade da estrutura do produto desenvolvido, utilizando no posible representacións gráficas.

Capítulo 5

Probas

Plan de probas (con evidencias) que verifica a funcionalidade e correctitude global do sistema, e se leva a cabo.

Capítulo 6

Exemplos (eliminar capítulo na versión final)

6.1. Un exemplo de sección

Esta é *letra cursiva*, esta é **letra negrilla**, esta é letra subrallada, e esta é **letra curier**. Letra tiny, scriptsize, small, large, Large, LARGE e moitas más. Exemplo de fórmula: $a = \int_0^\infty f(t)dt$. E agora unha ecuación aparte:

$$S = \sum_{i=0}^{N-1} a_i^2. \quad (6.1)$$

As ecuaciones se poden referenciar: ecuación (6.1).

6.1.1. Un exemplo de subsección

O texto vai aquí.

6.1.2. Outro exemplo de subsección

O texto vai aquí.

Un exemplo de subsubsección

O texto vai aquí.

Un exemplo de subsubsección

O texto vai aquí.

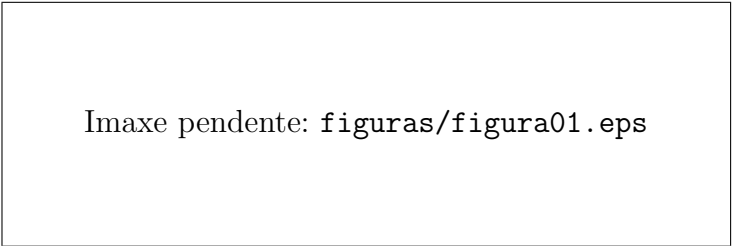


Figura 6.1: Esta é a figura de tal e cal.

Izquierda	Derecha	Centrado
ll	r	cccc
llll	rrr	c

Cadro 6.1: Esta é a táboa de tal e cal.

Un exemplo de subsubsección

O texto vai aquí.

6.2. Exemplos de figuras e cadros

A figura número 6.1.
O cadro (taboa) número 6.1.

6.3. Exemplos de referencias á bibliografía

Este é un exemplo de referencia a un documento descargado da web [1]. E este é un exemplo de referencia a unha páxina da wikipedia [2]. Agora un libro [3] e agora unha referencia a un artigo dunha revista [4]. Tamén se poden pór varias referencias á vez [1, 3].

6.4. Exemplos de enumeracións

Con puntos:

- Un.
- Dous.
- Tres.

Con números:

1. Catro.
2. Cinco.
3. Seis.

Exemplo de texto verbatim:

```
0 texto          verbatim
  se visualiza tal
    como se escribe
```

Exemplo de código C:

```
#include <math.h>
main()
{  int i, j, a[10];
   for(i=0;i<=10;i++) a[i]=i; // comentario 1
   if(a[1]==0) j=1; /* comentario 2 */
   else j=2;
}
```

Exemplo de código Java:

```
class HelloWorldApp {
    public static void main(String[] args) {
        System.out.println("Hello-World!"); // Display the string.
    }
}
```


Capítulo 7

Conclusións e posibles ampliacións

O traballo describe o grao de cumprimento dos obxectivos. Posibles vías de mellora.

Apéndice A

Manuais técnicos

En función do tipo de Traballo e metodoloxía empregada, o contido poderase dividir en varios documentos. En todo caso, neles incluírase toda a información precisa para aquelas persoas que se vaian encargar do desenvolvemento e/ou modificación do Sistema (por exemplo código fonte, recursos necesarios, operacións necesarias para modificacións e probas, posibles problemas, etc.). O código fonte poderase entregar en soporte informático en formatos PDF ou postscript.

Apéndice B

Manuais de usuario

Incluirán toda a información precisa para aquelas persoas que utilicen o Sistema: instalación, utilización, configuración, mensaxes de erro, etc. A documentación do usuario debe ser autocontida, é dicir, para o seu entendemento o usuario final non debe precisar da lectura doutro manual técnico.

Apéndice C

Licenza

Se se quere pór unha licenza (GNU GPL, Creative Commons, etc), o texto da licenza vai aquí.

Bibliografía

- [1] Nvidia CUDA programming guide. Versión 2.0, 2010. Disponible en <http://www.nvidia.com>.
- [2] Acceso múltiple por división de código. Artigo da wikipedia (<http://es.wikipedia.org>). Consultado o 2 de xaneiro do 2010.
- [3] R.C. Gonzalez e R.E. Woods, *Digital image processing*, 3ª edición, Prentice Hall, New York, 2007.
- [4] P. González, J.C. Cartex e T.F. Pelas, “Parallel computation of wavelet transforms using the lifting scheme”, *Journal of Supercomputing*, vol. 18, no. 4, pp. 141-152, junio 2001.